This script build the spreadsheets handed to the network and uploaded to maptiler

```
import numpy as np
import pandas as pd
import geopandas as gpd
import os.path as path
import utils
import glob
import re
import glob
from openpyxl import load_workbook, Workbook
from openpyxl.styles import Border, Side, Alignment
```

```
nuts_df = gpd.read_file(path.join(utils.raw_data_dir, "NUTS_RG_01M_2021_4326.shp"))
nuts_df = nuts_df.to_crs(epsg=3035) # convert to a metric CRS to compute surface area
nuts_df["area_km2"] = round(nuts_df.geometry.area / 1e6, 2)
```

```
# load population data from eurostat, including all sexes and ages
population_data = pd.read_csv(path.join(utils.raw_data_dir, "estat_demo_r_pjangrp3.tsv"))
population_data = population_data[population_data["sex"] == "T"]
population_data = population_data[population_data["age"] == "TOTAL"]
```

```
# due to the strange formatting of the .tsv file, i have to manually split the last column ir
last_col = population_data.columns[-1]

population_data[last_col] = population_data[last_col].apply(lambda x: [e for e in x.split("\t
population_data["NUTS_ID"] = population_data[last_col].apply(lambda x: x[0])
population_data["population"] = population_data[last_col].apply(lambda x: x[-1])
population_data["population"] = population_data["population"].apply(lambda x: re.sub(r"[a-zA-
population_data["population"] = population_data["population"].apply(lambda x: int(x) if len(x
population_data = population_data.drop(columns=[last_col, "sex", "age", "freq", "unit"])
```

```
# add population data to the nuts df and filter down to NUTS level 3
nuts_df = pd.merge(nuts_df, population_data, on="NUTS_ID", how="outer")
nuts3_df = nuts_df[nuts_df["LEVL_CODE"] == 3].drop(columns=["LEVL_CODE", "MOUNT_TYPE", "URBN_
```

```
# load previously computed cropland data
crop_profile_files = glob.glob(path.join(utils.intermediate_data_dir, "nuts3_crop_profile", "
crop_profile_df = pd.concat([gpd.read_file(f) for f in crop_profile_files if not f.lower().en
```

```
def calc_crop_area(in_dict):
    # computes cropland area from the number of pixels in the crop data for each crop type
    in_dict = eval(in_dict)
    out_dict = {}
    for k, v in in_dict.items():
        if k in [0, 65535]:
            # default value for non-cropland pixels is 0
            continue
        crop_name = utils.cropland_type_dict[k]
        # NOTE: this next step is important and deserves explanation
        # the raster size of the cropland dataset is _precisely_ 10x10m for all pixels, as de
        # thus, each pixel has an area of 100m^2. We get the total area in m^2 by multiplying
        # to get from m^2->km^2 we need to divide by 1.000*1.000, i.e. 1.000.000
        # in other words, we divide by 10.000 or 1e4
        area_km = round(v * 1e-4, 2)
        out_dict[crop_name] = area_km
    return out_dict

# compute crop area both total and by type for each NUTS-3 region
crop_profile_df["cropland_km2_by_type"] = crop_profile_df["crop_profile"].apply(calc_crop_are
crop_profile_df["cropland_km2"] = crop_profile_df["cropland_km2_by_type"].apply(lambda x: rou
```

```
# merge data into the main DataFrame
crop_profile_df = crop_profile_df[["NUTS_ID", "cropland_km2", "cropland_km2_by_type"]]
nuts3_df = pd.merge(nuts3_df, crop_profile_df, on="NUTS_ID", how="outer")
nuts3_df["cropland_area_percent"] = round(100 * nuts3_df["cropland_km2"] / nuts3_df["area_km2
```

```
nuts3_drought_data = utils.load_complex_geojson(
    path.join(utils.out_data_dir, "drought_days_nuts3.geojson")
)
nuts3_drought_data = nuts3_drought_data[
    [
        "NUTS_ID",
```

```
        "warning_days",
        "median_warning_days",
        "max_warning_days",
        "max_warning_days_year",
        "alert_days",
        "median_alert_days",
        "max_alert_days",
        "max_alert_days_year",
        "drought_days",
        "median_drought_days",
        "max_drought_days",
        "max_drought_days_year",
    ]
]
```

```
nuts3_df = gpd.pd.merge(nuts3_df, nuts3_drought_data, on="NUTS_ID", how="outer")
nuts3_df = nuts3_df.rename(columns = {colname:colname.lower() for colname in nuts3_df.columns
nuts3_df.set_index("nuts_id", drop=True, inplace=True)
nuts3_regions_outside_cdi_raster = nuts3_df[nuts3_df["drought_days"].isna()]
print(f"dropping {len(nuts3_regions_outside_cdi_raster)} rows from countries {set(nuts3_regio
nuts3_df = nuts3_df.dropna(subset="drought_days")
```

```
dropping 7 rows from countries {'NO', 'FR', 'PT'} where no drought data was present
```

```
year_cols = [f"Drought days {2012+i}" for i in range(14)]
nuts3_df[year_cols] = pd.DataFrame(nuts3_df["drought_days"].tolist(), index=nuts3_df.index)
```

```
# For both montenegro and Turkey, either LAU data or crop data is not present. So we drop all
eea_countries = [
    "BE",
    "BG",
    "CZ",
    "DK",
    "DE",
    "EE",
    "IE",
    "EL",
    "ES",
    "FR",
    "HR",
    "IT",
```

```
    "CY",
    "LV",
    "LT",
    "LU",
    "HU",
    "MT",
    "NL",
    "AT",
    "PL",
    "PT",
    "RO",
    "SI",
    "SK",
    "FI",
    "SE",
    "IS",
    "LI",
    "NO",
]
assert len(eea_countries) == 30

other_countries = [
    "UK", # uk, manually added
    "CH", # switzerland slovakia, albania, north macedonia are in the NUTS and LAU dataset, c
    "RS",
    "AL",
    "MK",
]
expected_countries = eea_countries + other_countries


a = list(set(nuts3_df["cntr_code"].values))
for b in a:
    if b not in expected_countries:
        print("dropping row for country: ", b)
nuts3_df = nuts3_df[nuts3_df["cntr_code"].isin(expected_countries)]
```

```
dropping row for country:  TR
dropping row for country:  ME
```

```
country_df = gpd.read_file(utils.country_dir)
groups = nuts3_df.groupby("cntr_code")
```

```
country_df = country_df[country_df["CNTR_ID"].isin( list(groups.groups.keys()))]
country_df = country_df.set_index("CNTR_ID")
```

```
for cntr, group_df in groups:
    country_dict = {}

    sum_columns = ["area_km2", "cropland_km2", "population"]
    for colname in sum_columns:
        if group_df[colname].isna().sum() == 0:
            country_dict[colname] = group_df[colname].sum()

    for drought_metric in ["warning", "alert", "drought"]:
        # for each of the three drought-indicator metrics, we compute the median and max for
        # read the metric days per year for each NUTS3 and store them in a 2d array of shape
        value_matrix = np.vstack(group_df[f"{drought_metric}_days"].values)
        # multiply the values for each NUTS3 region by the corresponding surface area in km2
        value_matrix = value_matrix * group_df["area_km2"].values.reshape(-1, 1)
        # sum over all nuts3 to get a 1d array with one value for each year.
        # Divide by the total surface area of the country to get areaa-weighted drought days
        value_matrix = value_matrix.sum(axis=0) / country_dict["area_km2"]

        # compute median, max and the year of the worst drought.
        country_dict[f"median_{drought_metric}_days"] = np.median(value_matrix).round(2)
        country_dict[f"max_{drought_metric}_days"] = np.max(value_matrix).round(2)
        country_dict[f"max_{drought_metric}_days_year"] = 2012 + np.argmax(value_matrix)

    for k, v in country_dict.items():
        country_df.loc[cntr, k] = v

    group_df = group_df[
        [
            "name_latn",
            "nuts_name",
            "area_km2",
            "cropland_area_percent",
            "population",
            "median_drought_days",
            "max_drought_days",
            "max_drought_days_year",
```

```
        ] + year_cols
    ]

    group_df.index.names = ["NUTS-3-ID"]
    group_df = group_df.rename(
        columns={
            "nuts_id": "NUTS-3-ID",
            "name_latn": "NUTS-3-Name / Name of geographical area",
            "nuts_name": "NUTS-3-Name (in English)",
            "median_drought_days": "Median drought days per year",
            "max_drought_days": "Maximum recorded drought days per year",
            "max_drought_days_year": "most severe drought year",
            "population": "Population",
            "area_km2": "Land area (square kilometers)",
            "cropland_area_percent": "Cropland as per centage of land area",
        }
    )
    country_name = country_df.loc[cntr, "NAME_ENGL"]
    out_dir = path.join(utils.out_data_dir, "country_spreadsheets", f"{country_name}.xlsx")
    group_df.to_excel(
        out_dir
    )
    print("saved to: ", out_dir)
```

```
import glob
from openpyxl import load_workbook, Workbook
from openpyxl.styles import Border, Side, Alignment

FILES = sorted(glob.glob(path.join(utils.out_data_dir, "country_spreadsheets", "*")))  # or l

COLUMN_WIDTHS = {
    "A": 10,
    "B": 20,
    "C": 20,
    "D": 15,
    "E": 15,
    "F": 15,
    "G": 20,
    "H": 20,
    "I": 15,
}
```

```
thin = Side(style="thin")
border = Border(left=thin, right=thin, top=thin, bottom=thin)

combined_wb = Workbook()
combined_wb.remove(combined_wb.active)  # remove default empty sheet

for filepath in FILES:
    wb = load_workbook(filepath)
    sheet_name = filepath.split("/")[-1].replace(".xlsx", "")[:31]  # Excel sheet name limit

    for src_ws in wb.worksheets:
        dest_ws = combined_wb.create_sheet(title=sheet_name)

        # Copy all cell values
        for row in src_ws.iter_rows():
            for cell in row:
                dest_ws[cell.coordinate].value = cell.value
                dest_ws[cell.coordinate].border = border
                dest_ws[cell.coordinate].alignment = Alignment(wrap_text=True, vertical="top

        for col_letter, width in COLUMN_WIDTHS.items():
            dest_ws.column_dimensions[col_letter].width = width

out_file = path.join(utils.out_data_dir, "country_stats.xlsx")
combined_wb.save(out_file)
print("Saved to ", out_file)
```

```
Saved to  /Users/johannesgille/Desktop/CE_2026_4_drought/data_out/country_stats.xlsx
```

```
country_df = country_df[['CNTR_NAME', 'NAME_ENGL', 'area_km2',
       'cropland_km2', 'population', 'median_warning_days', 'max_warning_days',
       'max_warning_days_year', 'median_alert_days', 'max_alert_days',
       'max_alert_days_year', 'median_drought_days', 'max_drought_days',
       'max_drought_days_year']]
country_df = country_df.rename({"CNTR_NAME": "cntr_id"})
```

```
country_df.to_json(path.join(utils.out_data_dir, "country_stats.geojson"), indent=2)
```

```
lau_df = utils.load_complex_geojson(path.join(utils.out_data_dir, "drought_days_lau.geojson")
lau_countries = set(lau_df["CNTR_CODE"].values)
```

```
nuts3_df = nuts3_df[nuts3_df["cntr_code"].isin(lau_countries)]
nuts3_df = nuts3_df.to_crs(epsg=4326)
nuts3_df.to_file(path.join(utils.out_data_dir, "nuts3_stats.geojson"))
```